

EECS 482

Introduction to operating systems

Distributed systems

Sami Fayaad
University of Michigan

From operating systems to distributed systems

- Why build distributed systems?
- People are distributed
 - Web, e-mail, SMS, social media, Google docs, Zoom
- Performance
 - Aggregate performance of many computers can be faster than that of (even a fast) single computer
- Reliability
 - Try to provide continuous service, even if some computers fail
 - Try to preserve data, even if some storage fails

Distributed systems are concurrent

- Concurrent programs need mechanisms for:
 - Synchronizing
 - Sharing information
- How did we accomplish these in multi-threaded programs?
- How should we accomplish these in a distributed system?

Using send/receive to share information

- Two styles of sharing information
 - Shared memory
 - Message passing

Using send/receive to synchronize

- How to provide mutual exclusion?
 - Need atomic primitive
 - In a distributed system, that atomic primitive is sending/receiving a network packet
 - How to build a larger critical section, e.g., sending multiple messages?
- How to provide ordering?
- Without shared data, are race conditions still possible?

Structuring a concurrent system

- One multi-threaded process on one computer
- Several multi-threaded processes, each running on a separate computer
- Several multi-threaded processes, all running on one computer.
 - Why do this?
 - Operating systems structured like this are called **microkernels**

Computer 1
Thread A
Thread B

Computer 1
Thread A
Thread B

Computer 2
Thread A
Thread B

Computer 1	
Process 1	Process 2
Thread 1A	Thread 2A
Thread 1B	Thread 2B

Administration

- Project 4
 - List slow operations
 - Test concurrency
 - Stress test
 - Test boundaries
 - Don't trust client input
- Schedule
 - Dec. 5: lab
 - Dec. 8: lecture
 - Dec. 17 @ 8:00-9:50 am: final exam
 - Dec. 17 @ 8:00 pm (online): post final exam lecture

Distributed file systems

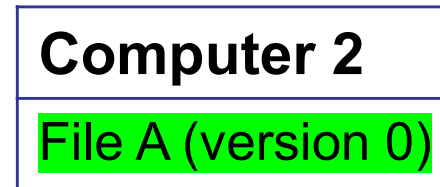
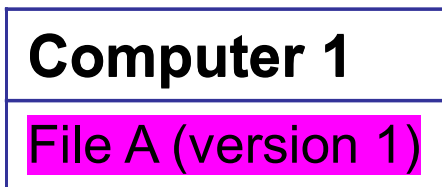
- Main abstraction: remote storage looks like local storage
- Examples?
- Basic implementation: a few clients, single server (Project 4)
- Advanced implementations: many clients, many servers
 - Client-side caching
 - Splitting data across multiple servers
 - ~~Replicating data across multiple servers~~

Client-side caching

- As more clients use a file server, what will be the bottleneck?
- One solution: add cache to each client
 - What does this improve?

Client-side caching

- Option 1: **one** copy of each file
 - Will this reduce server load and network traffic?
 - **Problems?**
- Option 2: allow multiple copies of each file
 - **Problems?**
 - Goal: present illusion of a single copy



Single-copy semantics

- Solution 1: broadcast changes

Computer 1
File A (version 1)

Computer 2
File A (version 1)

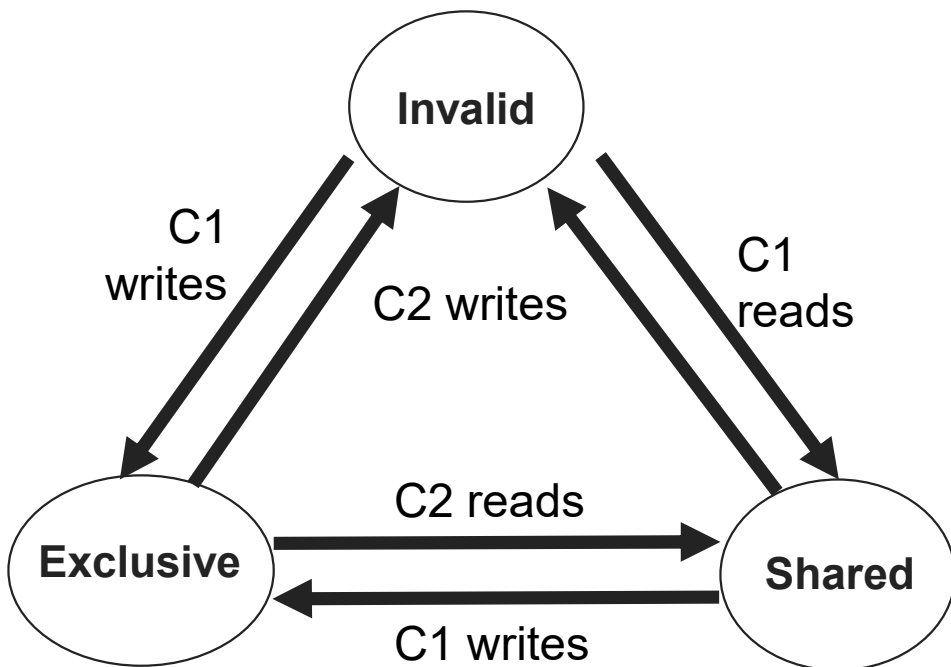
Single-copy semantics

MESI (modified/exclusive/
shared/invalid) protocols

- Solution 2: invalidate other copies

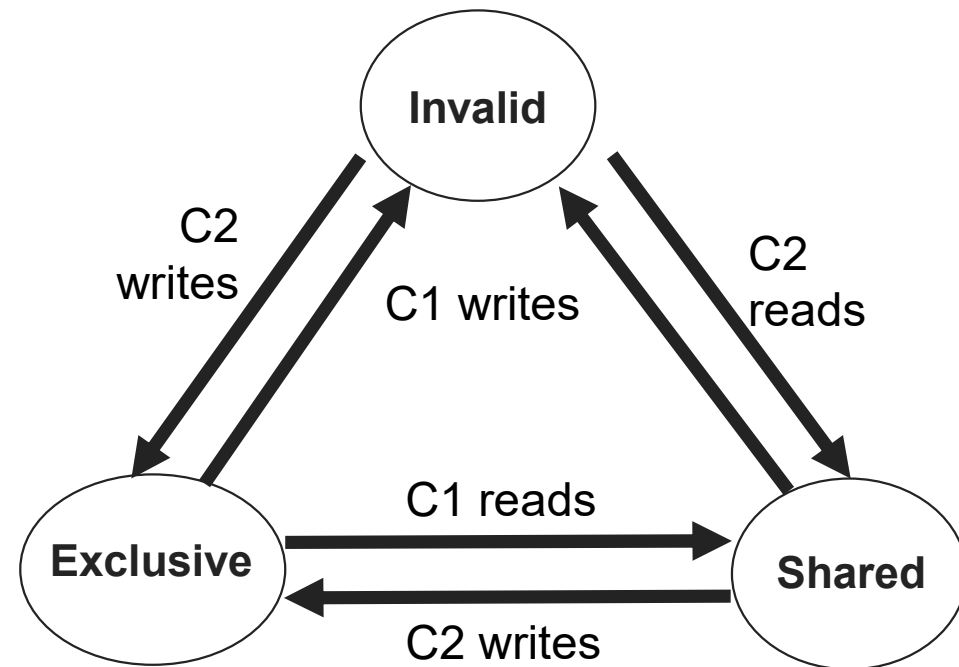
Computer 1

File A (version 2)



Computer 2

File A (version 2)



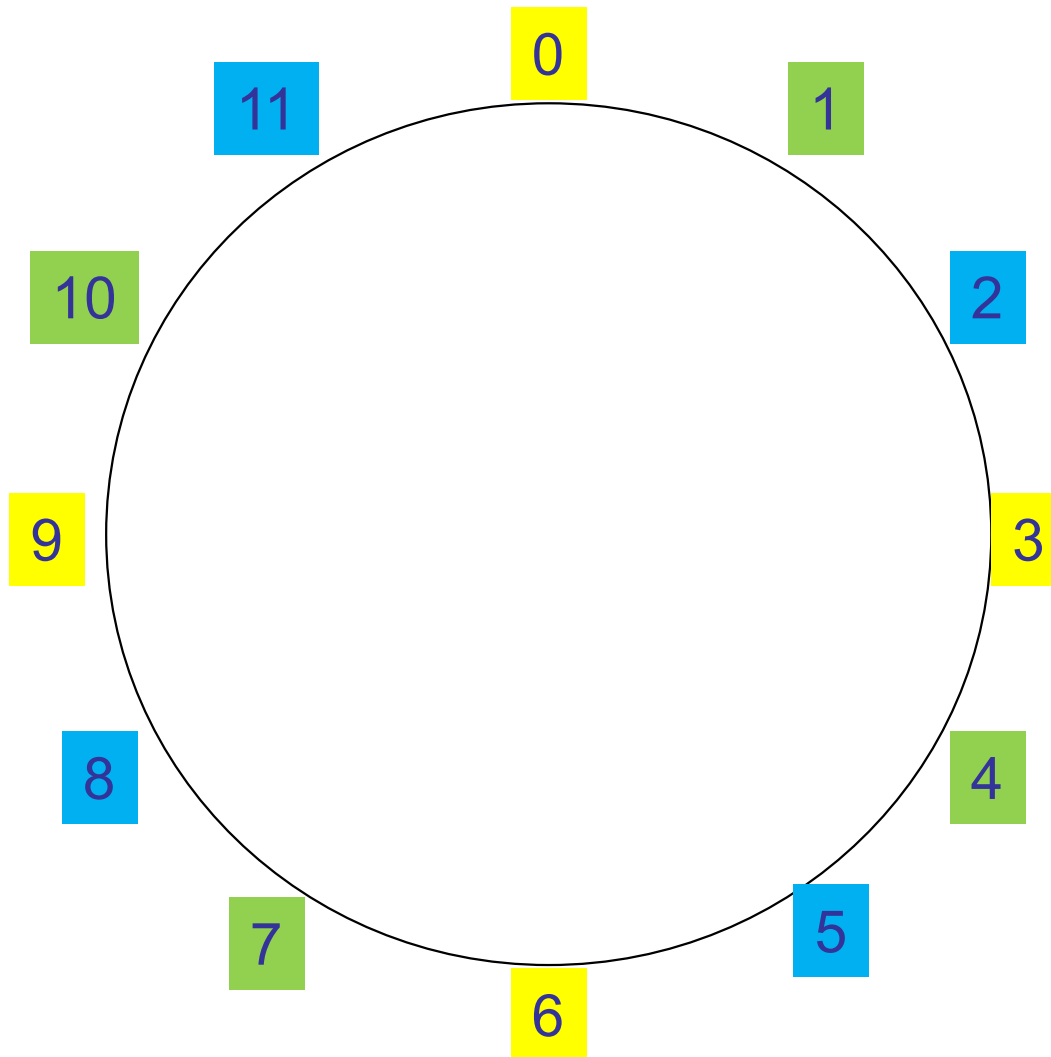
Multiple file servers (without replication)

- Assign different files to different servers
 - How to know which server to contact?

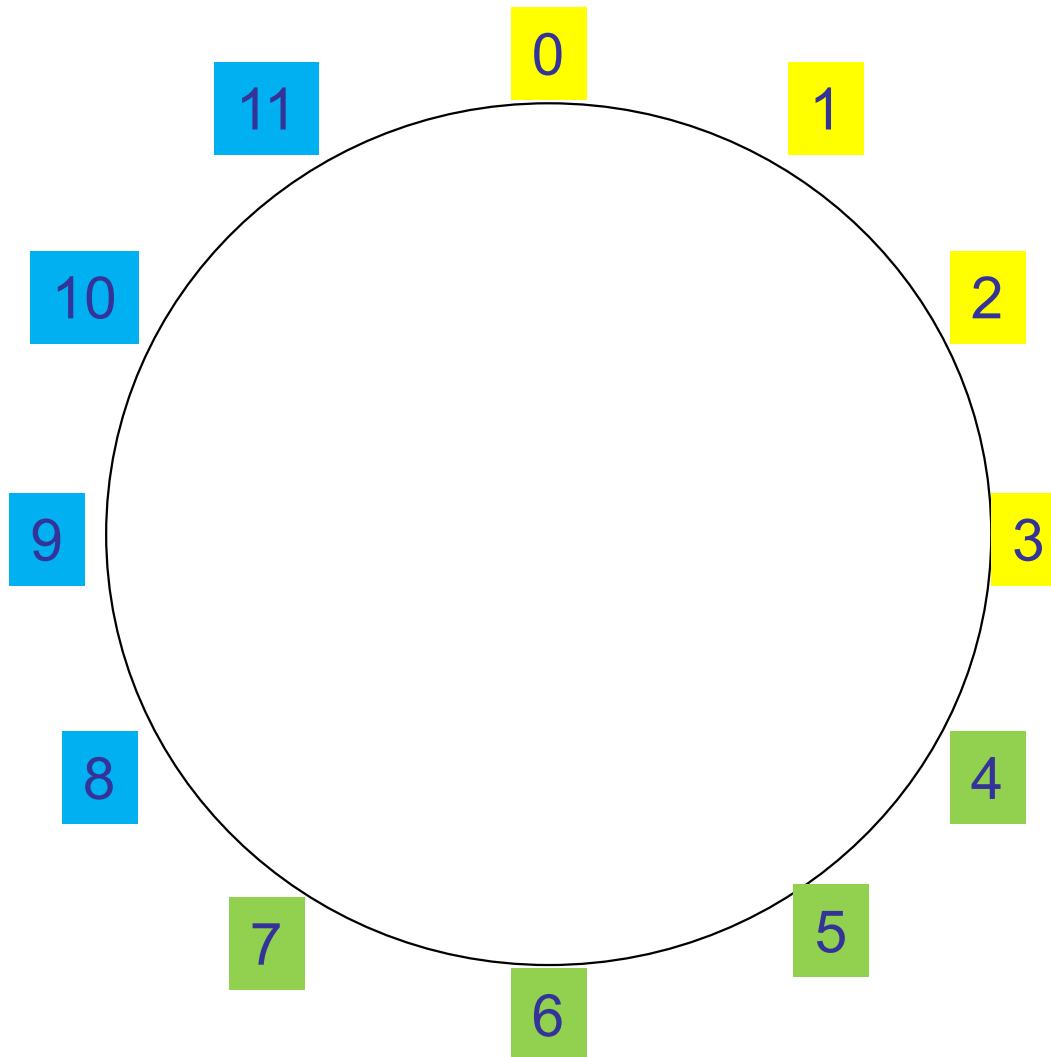
Hash-based partitioning

- Assign file f to server $[\text{hash}(f) \% n]$
- Problems?

Hash-based partitioning (modulus)



Hash-based partitioning (division)



Hash-based partitioning (random)

